AMERICAN INTERNATIONAL UNIVERSITY – BANGLADESH (AIUB)

Dept. of Computer Science

Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2024-2025

Section: [Your Section]

Project Report On

AgroBridge

An Agricultural Marketplace Management System

Supervised By

[Supervisor Name]

Submitted By:

Name	ID
[Your Full Name]	[Your Student ID]

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**

Evaluation Criteria	Evaluation Definition				Total = 15
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				**5**
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				**5**
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.				**5**

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				**Total = 15**
Organization of the application	The application demonstrates a well-structured, role-based desktop application with clear navigation and proper OOP design principles applied throughout.				**5**
Representation and Integration of Database	The system correctly integrates SQL Server Express with parameterized queries, transactions, and relational table design connecting all user roles and operations.				**5**
Graphical User Interface	The Windows Forms-based GUI provides intuitive, role-specific interfaces for Admin, Farmer, and Customer with proper validation and feedback messages.				**5**

Table of Contents

Chapter 1: Introduction

AgroBridge is a Windows Forms desktop application developed using C# and SQL Server Express as the backend database. The system serves as a digital agricultural marketplace that directly connects farmers with customers, eliminating the need for traditional middlemen. The application supports three distinct user roles — Admin, Farmer, and Customer — each with their own dedicated dashboard and set of functionalities.

The primary motivation behind AgroBridge is to empower local farmers by giving them a platform to list, manage, and sell their agricultural products directly to consumers. Customers can browse available products, add items to an in-memory shopping cart, and place orders which are processed using SQL transactions that guarantee stock consistency. The Admin manages user accounts and monitors overall sales through a dedicated reporting module.

The project demonstrates the application of Object-Oriented Programming (OOP) principles including encapsulation, abstraction, and separation of concerns through a three-tier architecture — UI Layer (Windows Forms), Business Logic Layer (Model classes), and Data Layer (SQL Server integration via SqlConnection, SqlCommand, SqlDataAdapter).

Chapter 2: Case Study

In Bangladesh, agricultural supply chains are heavily dependent on intermediaries, which drives up prices for consumers while reducing income for farmers. AgroBridge addresses this problem by providing a direct-to-consumer platform through a desktop application.

The system operates as follows: Farmers register on the platform and submit their profile for Admin approval. Once activated, they can list agricultural products with name, price, and stock quantity. Customers register separately, and upon activation can browse all available products, search by name, add items to a cart, adjust quantities, and place orders. When an order is placed, the system uses a SQL transaction to atomically record the order and deduct stock — ensuring no overselling occurs. The Admin oversees the entire platform, can activate or deactivate any user, delete accounts, and view a comprehensive sales report showing all orders with customer details and total revenue.

All new registrations default to **inactive status (Status = 0)**. This design choice ensures that only verified users can access the marketplace, providing a basic layer of platform security and quality control managed entirely by the Admin.

Key System Actors:

- **Admin:** Manages all registered users (activate, deactivate, delete). Views the full sales report showing orders, customer names, dates, and revenue totals.
- **Farmer:** Registers on the platform, lists agricultural products, updates prices and stock, removes own products. Cannot modify another farmer's products.
- **Customer:** Browses and searches available products, adds to cart, adjusts quantity, places orders. Order history viewable from profile.

Chapter 3: User Stories

3.1 Admin

• **Login:** The Admin logs in through FormLogin by entering username and password. The system performs a case-sensitive query against RegistrationTable. If credentials match and Status = 1 and Role = 'Admin', the Admin is directed to FormAdminHome.

• **View Registered Users:** From FormAdminHome, the Admin navigates to FormRegisteredUsers where all users (ID, Username, Name, Role, Status) are loaded into a DataGridView. The Admin can search by username or ID in real-time using DataView.RowFilter without additional database calls.

• **Activate / Deactivate User:** The Admin selects any row and clicks 'Update Status'. The system toggles the Status column between 0 (inactive) and 1 (active) in RegistrationTable using a parameterized UPDATE query.

• **Delete User:** The Admin enters a user's numeric ID and confirms deletion. The system runs a DELETE query on RegistrationTable. A confirmation dialog prevents accidental deletion.

• **View Sales Report:** The Admin can view a full sales report (SellsReport form / Form1.cs) that uses a LEFT JOIN between OrderTable and RegistrationTable to display every order with the customer name, date, item count, and total amount in BDT.

3.2 Farmer

• **Register:** A new Farmer registers via FormReg. After inserting into RegistrationTable, a matching row is also inserted into FarmerTable using the auto-generated ID (SCOPE_IDENTITY). Account starts as inactive.

• **Login:** Once activated by Admin, the Farmer logs in and is directed to FormFarmerHome where the farmerID is stored in the session.

• **Add Product:** From FormAddProduct, the Farmer enters a product name, price, and quantity. The system runs an INSERT INTO ProductTable with the logged-in farmerID linked as a foreign key.

• **Remove Product:** From FormRmvProduct, the Farmer enters a ProductID. The DELETE query includes AND FarmerID = @farmerID to enforce ownership — a farmer can only delete their own products.

• **Update Product:** From FormUpdateProduct, the Farmer searches by ProductID to verify ownership, then updates only the fields they modify using a dynamic UPDATE query. Ownership is checked both at UI level (FarmerID match) and DB level (AND FarmerID = @farmerID in WHERE clause).

• **View My Products:** FormViewProducts loads the farmer's products into a DataGridView using a SELECT filtered by FarmerID. A live search box uses LIKE filtering via SQL parameters.

3.3 Customer

• **Register:** A new Customer registers via FormReg. After inserting into RegistrationTable, a matching row is inserted into CustomerTable. Account starts inactive until Admin activates it.

• **Login:** Once activated, the Customer logs in. A Customer object is created with the logged-in CustomerID and Name and passed to FormCustomerHome.

• **Browse Products:** FormCustomerHome dynamically generates product cards in a FlowLayoutPanel showing product name, price, stock, and FarmerID. All products from ProductTable are fetched via Customer.LoadProducts().

• **Search Products:** The search box filters products in real-time using LINQ (.Where + .Contains) on the already-loaded product list — no extra DB query needed.

• **Add to Cart:** Clicking 'Add to Cart' on any product calls Customer.AddToCart(). Cart is an in-memory List<CartItem>. Duplicate products are rejected with an appropriate message.

• **Manage Cart:** FormCustomerCart shows all cart items with a NumericUpDown to adjust quantity. The total price updates live. The customer can remove individual items.

• **Place Order:** Clicking 'Order' in the cart triggers Order.PlaceOrder() which runs inside a SQL Transaction: inserts into OrderTable, then decrements stock in ProductTable for each item. If any item is out of stock, the whole transaction rolls back.

• **View Order History:** From the profile page, the Customer can navigate to FormCustomerOrders which loads all past orders (newest first) from OrderTable along with item details from OrderDetailsTable via a JOIN.

Chapter 4: Database Design

4.1 ER Diagram Description

The AgroBridge database (AgroBridgeDB) is hosted on SQL Server Express and consists of six interrelated tables. The central table is RegistrationTable which stores all users regardless of role. Farmers and Customers have dedicated profile tables (FarmerTable, CustomerTable) that hold a foreign key reference to RegistrationTable.ID. Products are stored in ProductTable with a FarmerID foreign key. Orders are split across OrderTable (header) and OrderDetailsTable (line items).

Relationships:

- RegistrationTable.ID (1) ■■< FarmerTable.FarmerID (N) — one registration entry maps to one farmer profile
- RegistrationTable.ID (1) ■■< CustomerTable.CustomerID (N) — one registration entry maps to one customer profile
- FarmerTable.FarmerID (1) ■■< ProductTable.FarmerID (N) — one farmer can have many products
- CustomerTable.CustomerID (1) ■■< OrderTable.CustomerID (N) — one customer can place many orders
- OrderTable.OrderID (1) ■■< OrderDetailsTable.OrderID (N) — one order contains many product line items
- ProductTable.ProductID (1) ■■< OrderDetailsTable.ProductID (N) — one product can appear in many orders

4.2 Table Schema

1. RegistrationTable

Column	Data Type	Constraint
ID	INT	PK, Identity, Auto Increment
Username	VARCHAR(100)	NOT NULL, UNIQUE
Password	VARCHAR(100)	NOT NULL
Name	VARCHAR(150)	NOT NULL
Role	VARCHAR(20)	NOT NULL ('Admin','Farmer','Customer')
Status	INT	NOT NULL, DEFAULT 0 (0=Inactive, 1=Active)

2. FarmerTable

Column	Data Type	Constraint
FarmerID	INT	PK, FK → RegistrationTable.ID

FName	VARCHAR(150)	NOT NULL
Username	VARCHAR(100)	NOT NULL
FContactInfo	VARCHAR(100)	NULLABLE
City	VARCHAR(100)	NULLABLE
Country	VARCHAR(100)	NULLABLE

3. CustomerTable

Column	Data Type	Constraint
CustomerID	INT	PK, FK → RegistrationTable.ID
CustomerName	VARCHAR(150)	NOT NULL
CustomerContactInfo	VARCHAR(100)	NULLABLE
City	VARCHAR(100)	NULLABLE
Country	VARCHAR(100)	NULLABLE

4. ProductTable

Column	Data Type	Constraint
ProductID	INT	PK, Identity, Auto Increment
ProductName	VARCHAR(150)	NOT NULL
Price	INT	NOT NULL
Quantity	INT	NOT NULL
FarmerID	INT	FK → FarmerTable.FarmerID

5. OrderTable

Column	Data Type	Constraint
OrderID	INT	PK, Identity, Auto Increment
CustomerID	INT	FK → CustomerTable.CustomerID
OrderDate	DATETIME	NOT NULL
TotalAmount	DECIMAL(10,2)	NOT NULL
Quantity	INT	Number of unique products ordered

6. OrderDetailsTable

Column	Data Type	Constraint

OrderDetailID	INT	PK, Identity, Auto Increment
OrderID	INT	FK → OrderTable.OrderID
ProductID	INT	FK → ProductTable.ProductID
Quantity	INT	Quantity of this product in the order

4.3 SQL Queries Used in the Project

Login Authentication (FormLogin.cs)

```
SELECT * FROM RegistrationTable WHERE Username = 'value' COLLATE
SQL_Latin1_General_CP1_CS_AS AND Password = 'value' COLLATE
SQL_Latin1_General_CP1_CS_AS;
```

Purpose: Verifies credentials with case-sensitive collation. Checks Status and Role columns from the result.

Check Duplicate Username (FormReg.cs)

```
SELECT * FROM RegistrationTable WHERE Username COLLATE
SQL_Latin1_General_CP1_CS_AS = @username
```

Purpose: Prevents duplicate registrations. Uses parameterized query.

Register New User (FormReg.cs)

```
INSERT INTO RegistrationTable (Username, Password, Name, Role, Status) VALUES
(@username, @password, @name, @role, 0); SELECT CAST(SCOPE_IDENTITY() AS INT);
```

Purpose: Inserts the new user with Status=0 (inactive). SCOPE_IDENTITY() returns the auto-generated ID for linking to FarmerTable or CustomerTable.

Insert Farmer Profile (FormReg.cs)

```
INSERT INTO FarmerTable (FarmerID, FName, Username, FContactInfo, City,
Country) VALUES (@id, @name, @username, NULL, NULL, NULL)
```

Purpose: Creates farmer profile using the same ID from SCOPE_IDENTITY().

Load All Products (Customer.cs)

```
SELECT ProductID, ProductName, Price, Quantity, FarmerID FROM ProductTable
```

Purpose: Loads all products to display on the customer home page. Read with SqlDataReader into List.

Add Product (Farmer.cs)

```
INSERT INTO ProductTable (ProductName, Price, Quantity, FarmerID) VALUES
(@pname, @price, @qty, @farmerID)
```

Purpose: Inserts a new product linked to the logged-in farmer.

Remove Product (Farmer.cs)

```
DELETE FROM ProductTable WHERE ProductID = @pid AND FarmerID = @farmerID
```

Purpose: Ownership check via AND FarmerID clause. Farmers can only delete their own products.

Update Product - Dynamic (Farmer.cs)

```
UPDATE ProductTable SET [only changed fields] WHERE ProductID = @pid AND
FarmerID = @farmerID
```

Purpose: Dynamic SET clause built at runtime. Only provided fields are updated. Ownership enforced in WHERE.

Get Farmer's Products with Filter (Farmer.cs)

```
SELECT ProductID, ProductName, Price, Quantity FROM ProductTable WHERE
FarmerID = @farmerID [AND ProductName LIKE @pname]
```

Purpose: Optional LIKE filter for live search in FormViewProducts. Returns DataTable via SqlDataAdapter.

Place Order - Transaction (Order.cs)

```
INSERT INTO OrderTable (CustomerID, OrderDate, TotalAmount, Quantity) OUTPUT
INSERTED.OrderID VALUES (@CustomerID, @OrderDate, @TotalAmount, @Quantity)
```

Purpose: Inside a SqlTransaction. OUTPUT INSERTED.OrderID returns the new OrderID without a second query.

Decrement Stock on Order (Order.cs)

```
UPDATE ProductTable SET Quantity = Quantity - @Qty WHERE ProductID =
@ProductID AND Quantity >= @Qty
```

Purpose: Atomic stock deduction. AND Quantity >= @Qty prevents negative stock. Returns 0 rows if stock insufficient, triggering Rollback().

Get Customer Orders (Order.cs)

```
SELECT OrderID, OrderDate, TotalAmount, Quantity FROM OrderTable WHERE
CustomerID = @CustomerID ORDER BY OrderDate DESC
```

Purpose: Fetches full order history for the customer, newest first.

Get Order Item Details - JOIN (Order.cs)

```
SELECT od.ProductID, p.ProductName, p.Price, od.Quantity FROM
OrderDetailsTable od JOIN ProductTable p ON od.ProductID = p.ProductID WHERE
od.OrderID = @OrderID
```

Purpose: Joins OrderDetailsTable with ProductTable to retrieve product names for each order.

Toggle User Status (FormRegisteredUsers.cs)

```
UPDATE RegistrationTable SET Status = @Status WHERE Username = @Username
```

Purpose: Admin toggles between 0 and 1. New value determined in C# before the query runs.

Delete User (FormRegisteredUsers.cs)

```
DELETE FROM RegistrationTable WHERE ID = @ID
```

Purpose: Admin permanently removes a user. Confirmation dialog required before execution.

Sales Report - JOIN (Form1.cs / SellsReport)

```
SELECT o.OrderID, o.OrderDate, o.TotalAmount, o.Quantity, r.Name AS
CustomerName, r.Username FROM OrderTable o LEFT JOIN RegistrationTable r ON
o.CustomerID = r.ID ORDER BY o.OrderDate DESC
```

Purpose: LEFT JOIN ensures orders still appear even if the customer account was deleted (shown as 'Unknown').

Chapter 5: System Architecture & Class Design

The AgroBridge application follows a loosely structured three-tier architecture, implemented as a single C# Windows Forms project (.NET Framework 4.7.2).

• **UI Layer (Presentation):** All Form*.cs files. Each form is responsible only for displaying data and capturing user input. Forms receive data through constructor parameters (e.g., Customer object, farmerID) and delegate all business logic to the model classes.

• **Business Logic Layer:** Customer.cs, Farmer.cs, Order.cs. These classes contain all data processing logic, validation, and SQL operations. They hold the database connection string and expose clean methods like AddProduct(), PlaceOrder(), SearchProducts() to the UI layer.

• **Data Layer:** SQL Server Express (AgroBridgeDB). Accessed exclusively through System.Data.SqlClient — SqlConnection, SqlCommand, SqlDataReader, SqlDataAdapter, and SqlTransaction.

5.1 Model Classes

Class	Role	Key Methods / Properties
Product.cs	Data model	ProductID, ProductName, Price, Quantity, FarmerID (auto-properties only, no DB logic)
CartItem.cs	Data model	ProductID, ProductName, Price, Quantity, TotalPrice (computed). In-memory only.
Customer.cs	Business logic	LoadProducts(), SearchProducts(), AddToCart(), RemoveFromCart(), ClearCart(). Holds List<
Farmer.cs	Business logic	AddProduct(), RemoveProduct(), UpdateProduct(), GetMyProducts(). All methods use parame
Order.cs	Business logic	PlaceOrder() with SqlTransaction, GetOrdersByCustomer() (static), GetTotalAmount(). Links

5.2 Form Classes (UI Layer)

Form File	Purpose	DB Connection?
Program.cs	Entry point. Launches FormWelcome.	No
FormWelcome.cs	Welcome screen. Navigates to Login or Register.	No
FormLogin.cs	Authenticates user. Routes to role-specific home form.	Yes (direct SqlConnection)
FormReg.cs	New user registration. Inserts into RegistrationTable + role table.	Yes (direct SqlConnection)
FormChangePass.cs	Password change. Verifies old password, updates new.	Yes (direct SqlConnection)
FormAdminHome.cs	Admin dashboard. Navigates to user management and reports.	No
FormRegisteredUsers.cs	Admin: view, search, activate, deactivate, delete users.	Yes (direct SqlConnection)
Form1.cs (SellsReport)	Admin: full sales report using LEFT JOIN query.	Yes (direct SqlConnection)
FormFarmerHome.cs	Farmer dashboard. Navigates to product management forms.	No
FormAddProduct.cs	Farmer: add new product via Farmer.AddProduct().	Via Farmer class
FormRmvProduct.cs	Farmer: remove product by ID via Farmer.RemoveProduct().	Via Farmer class

FormUpdateProduct.cs	Farmer: search product, verify ownership, update details.	Yes + Via Farmer class
FormViewProducts.cs	Farmer: view and search own products in DataGridView.	Via Farmer class
FormCustomerHome.cs	Customer: browse all products as cards, add to cart, search.	Via Customer class
FormCustomerCart.cs	Customer: view cart, adjust quantity, place order.	Via Order class
FormCustomerProfile.cs	Customer: view profile, navigate to order history.	No
FormCustomerOrders.cs	Customer: view full order history loaded from DB.	Yes (direct SqlConnection)

5.3 Connection String

The same connection string is used across all classes and forms that connect to the database:

```
Data Source=.\SQLEXPRESS;Initial Catalog=AgroBridgeDB;Integrated
Security=True;Encrypt=False;TrustServerCertificate=True
```

• **.\SQLEXPRESS** — connects to the local SQL Server Express named instance on the same machine.

• **AgroBridgeDB** — the database name (restored from AgroBridgeDB.bak file included in submission).

• **Integrated Security=True** — uses Windows Authentication. No username or password is hardcoded in source code.

• **Encrypt=False / TrustServerCertificate=True** — disables SSL certificate validation for local development environment to prevent connection errors.

Chapter 6: Functionalities

Admin Functionalities:

- Login with case-sensitive credential verification
- View all registered users in a DataGridView (ID, Username, Name, Role, Status)
- Real-time search of users by username or ID (DataView.RowFilter — no extra DB call)
- Toggle user account status between Active (1) and Inactive (0)
- Delete any user account by entering their numeric ID with confirmation dialog
- View full sales report: all orders with customer name, date, item count, and total BDT amount

Farmer Functionalities:

- Register as a Farmer (account activated by Admin before first login)
- Add new agricultural products with name, price, and quantity
- Remove own products by ProductID — ownership enforced at DB level
- Update product details (name, price, quantity) — dynamic SQL, only changed fields updated
- View own product list in DataGridView with live search by product name
- Change password from the login screen

Customer Functionalities:

- Register as a Customer (account activated by Admin before first login)
- Browse all available products displayed as dynamic UI cards with name, price, stock, and FarmerID
- Live product search using LINQ on the in-memory product list
- Add products to an in-memory shopping cart — duplicate detection prevents double-adding
- Adjust product quantity in cart using NumericUpDown control — total price updates live
- Remove individual items from the cart
- Place order with SQL Transaction — atomically inserts order record and decrements product stock
- Insufficient stock detection — if any product is out of stock, the entire order is rolled back
- View order history showing all past orders with date, total amount, and item details
- Change password from the login screen

Key Technical Features:

- **SQL Transaction in PlaceOrder():** Ensures atomicity — either all stock updates succeed or everything rolls back. Prevents partial orders and negative stock.

• **OUTPUT INSERTED.OrderID:** Retrieves auto-generated primary key in the same INSERT statement using ExecuteScalar(), avoiding a second roundtrip to the database.

• **Dynamic UPDATE query in UpdateProduct():** Builds the SET clause at runtime based on which fields the user actually filled in, avoiding unnecessary overwrites.

• **Ownership enforcement:** All DELETE and UPDATE queries on ProductTable include AND FarmerID = @farmerID in the WHERE clause, providing DB-level row authorization.

• **In-memory cart with session persistence:** The Customer object (including its Cart list) is created at login time and passed between forms as a constructor parameter, keeping cart state alive across navigation.

• **COLLATE SQL_Latin1_General_CP1_CS_AS:** Applied on login and registration username checks to enforce case-sensitive comparison, preventing username impersonation.

• **Role-based routing at login:** FormLogin checks the Role column and directs users to Admin, Farmer, or Customer dashboards respectively.

• **Status-based access control:** All new registrations default to Status=0 (inactive). Login is blocked for inactive accounts with a clear message to contact Admin.

Chapter 7: Screenshots of Forms

The following section describes each form in the AgroBridge application. Please attach actual screenshots from the running application here.

FormWelcome — Welcome Screen

The entry point of the application. Displays the AgroBridge title and two buttons: 'Login' and 'Register'. Closing this form exits the application.

[Screenshot Placeholder — Attach actual screenshot here]

FormLogin — Login Screen

Accepts username and password. Case-sensitive authentication via COLLATE. Routes to Admin, Farmer, or Customer dashboard based on Role column. Also provides 'Change Password' navigation.

[Screenshot Placeholder — Attach actual screenshot here]

FormReg — Registration Form

Accepts first name, last name, username, password, gender, age, and role (Farmer/Customer). Checks for duplicate usernames before inserting. New accounts default to inactive status.

[Screenshot Placeholder — Attach actual screenshot here]

FormChangePass — Change Password

Accepts current username, old password, new password, and confirmation. Verifies old credentials before updating. Returns to login on success.

FormAdminHome — Admin Dashboard

Central navigation hub for Admin. Two primary options: View Registered Users and View Sales Report.

FormRegisteredUsers — User Management

DataGridView showing all users with ID, Username, Name, Role, Status. Live search by username or ID. Buttons for toggling status and deleting users.

SellsReport (Form1) — Sales Report

Text-based report showing all orders with OrderID, customer name, username, date, item count, and total BDT. Grand total and order count displayed at the bottom.

FormFarmerHome — Farmer Dashboard

Navigation hub for Farmer role with four buttons: Add Product, Remove Product, Update Product, View My Products.

FormAddProduct — Add Product

Three text fields for product name, price, and quantity. Validation ensures no empty fields. On success, calls Farmer.AddProduct() which runs the INSERT query.

FormRmvProduct — Remove Product

Single text field for ProductID. Calls Farmer.RemoveProduct() which runs DELETE with ownership check.

FormUpdateProduct — Update Product

Search by ProductID to load current data into fields. Verifies ownership before showing. Update button calls Farmer.UpdateProduct() with dynamic SQL.

FormViewProducts — View My Products

DataGridView showing the farmer's own products. Live search box filters by product name using LIKE query.

FormCustomerHome — Customer Home

FlowLayoutPanel with dynamically generated product cards showing name, price, stock, FarmerID, and 'Add to Cart' button. Search box filters in real time.

FormCustomerCart — Shopping Cart

Lists all cart items with product name, price, ProductID, and quantity selector. Total price updates live. 'Order' button triggers the SQL transaction.

FormCustomerProfile — Customer Profile

Simple profile page with customer name and a button to view order history.

FormCustomerOrders — Order History

FlowLayoutPanel showing all past orders with OrderID, date (formatted as dd MMM yyyy, hh:mm tt), and total amount in BDT. Newest orders shown first.

Chapter 8: Conclusion

AgroBridge successfully demonstrates a complete, real-world desktop application built with C# Windows Forms and SQL Server Express. The system addresses the practical problem of disconnection between farmers and consumers in Bangladesh by providing a structured digital marketplace with role-based access control, transactional data integrity, and a user-friendly interface.

From a technical standpoint, the project applies core Object-Oriented Programming concepts throughout: encapsulation is demonstrated in the Customer, Farmer, and Order model classes which hide their database logic behind clean public methods; abstraction is achieved by separating UI concerns from business logic; and the three-tier architectural pattern (UI → Business Logic → Database) makes the codebase organized and maintainable.

The use of SQL Transactions in the order placement module is a highlight of the project, ensuring that stock deductions and order records are always consistent — a critical requirement in any e-commerce system. The dynamic UPDATE query in the product management module demonstrates practical database programming skills beyond basic CRUD operations.

While the project achieves all specified requirements, there are areas identified for future improvement: full parameterization of all SQL queries (particularly in FormLogin and FormChangePass where string concatenation currently exists), implementation of password hashing for security, pagination for large product lists, and potential migration to a multi-tier web-based architecture for broader accessibility.

This project represents a meaningful application of the skills developed throughout the CSC2210 Object Oriented Programming 2 course, combining C# GUI development, relational database design, and software engineering principles into a functional and deployable desktop system.

— End of Report —